

APE: Dynamic Workload Management Module for Apache AsterixDB

Status

dev@asterixdb.a.o thread	(permalink to discuss thread)
JIRA issue(s)	https://issues.apache.org/jira/projects/ASTERIXDB/issues/...
Release target	N/A

Members

- Shiva Jahangiri
- Hongyu Shi

Motivation

Overview:

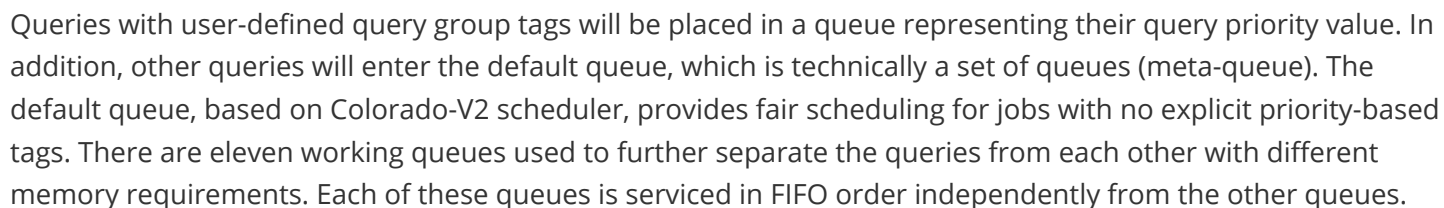
The old FIFO scheduler for query jobs has several limitations, such as treating all jobs with equal priority, which leads to resource-intensive jobs blocking shorter ones under the basic FIFO policy. Moreover, users have limited control over the execution order of queries, despite the common need for clients to manage queries with different levels of importance.

To resolve these issues, the new module enables more robust and stable query execution by allowing the system to prioritize queries according to user-defined classifications and priorities. It ensures that short, low-resource queries are prioritized, while also maintaining fair scheduling by balancing wait times relative to execution times.

Design:

- **Architecture**

Users can define query groups along with their respective priority levels. Priority values are assigned as integers starting from 1, where 1 represents the lowest priority. AsterixDB stores metadata about these query groups and their priorities. Each query can have one of three possible tags: a user-configured query group, a "short" tag, or a "long" tag, and it also may have no tags at all.



Q_Tag = Long



Mem_Req $\leq 0.05 \times \text{Total Mem}$



$0.6 \times \text{Total Mem} \leq \text{Mem_Req} \leq 0.10 \times \text{Total Mem}$



.....

$0.85 \times \text{Total Mem} \leq \text{Mem_Req} \leq \text{Total Mem}$



Colorado-V2's study showed that mixing zero-memory ($\text{mem_req} \leq 0.05 \times \text{Total Memory}$) short and long queries in the same queue due to similar memory needs can degrade short query performance. Therefore, long queries, identified through cost functions or user hints ("long" tag), are placed in a separate working queue within the fair scheduler, not to affect other queries with similar resource requirements but significantly different execution times.

Upon arrival of a new query or completion of an existing query, all the queries at the head of each queue are evaluated to be considered as its next query for execution. In this evaluation, we measure the slowdown of the queries at the head of the queues, the query with the highest slowdown value is selected as the next query to execute. The slowdown is defined as:

$$\text{slowdown} = (\text{waitTime} + \text{averageExecTime}_{\text{class}}) / \text{averageExecTime}_{\text{class}}$$

The average execution time of a query class is used because the current version of AsterixDB lacks cost-based optimizer functions or other prediction mechanisms based on the query plan to estimate execution time (The calculation is described in the next execution).

- **Initialization and Admission**

Upon starting AsterixDB with the configured settings, it initializes the queues based on the provided configurations. Users have the flexibility to adjust the priority of query groups, allowing them to dynamically reorder and prioritize queries during runtime.

Short Query Accelaration: Short queries will use their own dedicated resources and their queries are selected in FIFO manner independently from other queries and queues.

Priority-Based Scheduling: In order to select the next query from the prioritize-based queues, the scheduler first selects the queues with at least one query waiting in them and creates a set from their priorities (eligible set). Next, it can select the best query based on Probability Distribution. In this approach, the system will assign a priority distribution to each priority-based queue according to the following equation:

$$\frac{P_{Q_i}}{\sum_{j=1}^N P_{Q_j}}$$

After assigning the priority distribution for each queue, a random number generator will produce a number between 0 and 1 which selects the next queue to be selected as target queue. This process happens after each query selection. In the scenario that the system does not have enough resources for executing the selected query, two approaches can be considered: 1) Generate another random queue priority number 2) Store the selected query in a variable and wait for it to get executed.

Our scheduler exclusively determines query execution order. Meanwhile, except for the short query queue, no other queue has dedicated resources or resource quota. Instead, each selected query receives the amount of resources defined by its (default or user-defined) memory and CPU budget values, consistent with the current design of AsterixDB.

The default queue, which ensures fair scheduling for those without specific user-defined class tags, functions similarly to a priority-based queue during the job selection phase. Users can also adjust the priority of the entire default queue component. As a result, job selection based on probability may yield the default queue, after which the query with the highest slowdown of all working queues inside is selected for execution.

Calculation of average execution time: Each memory requirement class's working queue begins with an offline value for the average execution time, while the query count (q_count) starts at zero at the start-up phase. When the first query is executed from a queue, if the q_count is zero, the average is set to the execution time of that query, and the count is incremented to one. After that, the Exponentially Weighted Moving Average (EWMA) strategy is applied to update the average execution time. A parameter β is used to control how much importance is placed on the latest execution in the calculation.

$$\begin{aligned} \beta &= \text{Weight Parameter} \\ \theta_R &= \text{Rth round Execution Time} \\ \theta_0 &= \text{Execution Time at Startup Phase} \\ \mathbf{W}_R &= \text{Rth round EWMA} \end{aligned} \tag{1}$$

To calculate the current EWMA for a working queue, the most recent execution time is combined with previous data, with older values decaying over time. In general, to compute Rth round EWMA for a given weight parameter β , we have:

$$\mathbf{W}_R = \begin{cases} \theta_0, & R = 0 \\ \beta \cdot \mathbf{W}_{R-1} + (1 - \beta) \cdot \theta_R, & R > 0 \end{cases} \tag{2}$$

A higher β value reduces the impact of new data on the average, as the weight of older execution times gradually diminishes.

Proposed changes

Interface:

- Configure query group: An openly typed metadata collection is introduced for defining query groups and their priorities. One approach taking input is to enable users update query group information or modify its priority via common DML interface. A variation is to manipulate the information via new DDL syntax rather than by operating explicitly on the collection. This feature is discussed in another APE proposal.
- Statements indicating query groups and hints:
 - " SET `querygroup` `{group\_name}`;" assigns a specific pre-defined query group for the query.
 - " SET `querygroup` 'short';" and " SET `querygroup` 'long';" marks the query with 'short' and 'long' tags utilized by the scheduler

Changes:

The `ICapacityController`, which offers a resource allocation interface based on query job specifications, is added with an additional interface to inform the maximum available aggregated resources in the current cluster to external components.

Compatibility/Migration

The proposed changes don't modify any existing flow or behavior and can be seen as an extension. To enable or disable dynamic work management, the parameter `job.manager.class` and `job.queue.class` can be configured in the [cc] section of the cc.conf file. The system use the original job manager and FIFO queue by default.

Testing

Correctness:

- Unit Test
 - Components of priority-based queues, default job queue, dedicated resource queue.
 - Dynamic job manager
 - Resource controller guard for different queues.
- Verify policies under different mixed workloads
 - The scheduler act as a fair scheduler if the user does not define any user-defined query tag.
 - The scheduler act as a fully prioritized scheduler when the user uses user-defined query tags as no query will enter the default queue.
 - The scheduler can act as a hybrid scheduler when user uses user-defined query tags for some queries and no tags for other queries. In this case, the scheduler will act as a fair scheduler for non-tag queries and as a prioritized scheduler for the queries with user- defined query tags.

Performance:

To run benchmark experiment, we use AsterixDB's scheduler Workload Generator (<https://github.com/shivajah/SchedulerWorkloadGenerator>), which utilizes data generated from Wisconsin JSON Data Generator(<https://jimgray.azurewebsites.net/benchmarkhandbook/chapter4.pdf>)